

GOOGLE × KAGGLE · 5-DAY AI AGENTS INTENSIVE (VIBE CODING)

WaterWatch

The AI agent that doesn't just tell you your water is unsafe — it proves it with receipts, then files, tracks, and escalates on your behalf.

A verifiable, civic-action multi-agent system · Track: Agents for Good

TRACK

Agents for Good

judges may re-slot winners

TEAM

2 builders

roles defined inside

STACK

ADK · Gemini 3

MCP · A2A · GCP live

DEADLINE

Jul 6, 2026

submit early

Executive Summary

WaterWatch is a multi-agent system that turns an unreadable water-quality lab report into a verified safety verdict and a concrete, accountable action — for the millions of Indian households, Resident Welfare Associations (RWAs), and gram panchayats who today have no way to know whether their water is safe or to make anyone fix it.

A person uploads a lab report (PDF or phone photo). A team of specialized Gemini-powered agents reads it, checks every parameter against the BIS 10500 drinking-water standard, cross-references the area's official readings from India's open-government water-quality data, flags exactly which contaminants are dangerous and why, recommends the cheapest filtration matched to those specific contaminants, drafts a complaint to the water board, tracks whether it gets resolved, and — if it is ignored — drafts a Right to Information (RTI) escalation. Every single claim it makes carries a citation receipt back to an authoritative source, so it is structurally incapable of inventing a safe limit or a health risk.

WHY THIS WINS, IN ONE LINE

The model is not the differentiator — almost no one's is. WaterWatch wins on three things judges can see and most submissions lack: **verifiability by design** (citation receipts + a self-checking verifier agent), **real actuation** (it files and escalates, it doesn't just chat), and an **evaluation harness** (the AgentOps discipline from Day 4 that nearly every capstone skips).

Contents

1	The Problem	3
2	The Solution & Its Core Principle	4
3	Why Agents — Not a Chatbot	5
4	System Architecture (with diagram)	6
5	The MCP Data Layer	9
6	Technology Stack	10
7	Live GCP Deployment Architecture	11
8	Security & Responsible-AI Design	13
9	How Course Days 1–5 Build This Project	14
10	Winning Strategy: Scoring the Rubric	15
11	Demo & 5-Minute Video Plan	17
12	Team Split (2 builders)	18
13	Risks & Mitigations	18
14	Advanced Future Roadmap	19
A	Appendix A — README.md skeleton	20

The Problem

Knowing whether your drinking water is safe — and getting it fixed when it isn't — is broken at every step for ordinary people in India.

Four failures stacked on top of each other

1. The report is unreadable.

A standard water test returns 14+ parameters — pH, turbidity, TDS, total hardness, chloride, fluoride, nitrate (NO_3^-), arsenic (As), lead (Pb), iron (Fe), total coliforms, *E. coli*, and more. To a resident, an RWA secretary, or a panchayat member, the numbers are meaningless. "Fluoride: 1.8 mg/L" looks fine until you know the limit is 1.0.

2. The naive AI fix is dangerous.

The obvious move — "paste it into a chatbot" — is exactly the wrong one for a *health* decision. A language model will confidently state a safe limit it half-remembers, or invent a health effect. When the output drives whether a family keeps drinking the water, a hallucinated threshold is not a cosmetic bug; it is a safety failure.

3. There is no accountability loop.

Even when a resident *knows* the water is contaminated, the path to action is opaque: which authority, what format, what to cite, and how to follow up. Most complaints are never filed, and the ones that are simply vanish.

4. The civic signal is lost.

When ten households in the same pincode all have high nitrate, that is a public-health pattern that should trigger municipal action. Today those ten reports never meet each other; each family suffers alone.

THE OPPORTUNITY

The ground truth is fixed and public (the BIS 10500 standard), the area data is openly available (India's Open Government Data platform and CPCB), and the actions are well-defined (municipal complaint, RTI). Everything needed to close this loop exists — it has just never been *assembled into a single agent that can reason over it and act*. That assembly is WaterWatch.

The Solution & Its Core Principle

WaterWatch is a "verifiable civic-action agent." It is built around a single non-negotiable principle that shapes the entire architecture: **the agent may never assert a number or a health claim it cannot cite.**

What it does, end to end

1. **Reads** the uploaded lab report with Gemini's multimodal vision and extracts every parameter into structured data — no manual typing.
2. **Checks** each value against BIS 10500 acceptable and permissible limits, returning exactly which parameters breach and by how much.
3. **Contextualizes** by pulling the area's recent official water-quality readings from open-government data, so the user sees how their sample compares to what the municipality reports.
4. **Explains** the health impact of each breached contaminant, grounded in WHO/BIS references, with a citation on every statement.
5. **Verifies** itself: a dedicated agent rejects any line of the report that lacks a citation receipt, looping until the output is clean.
6. **Recommends** the cheapest filtration matched to the *specific* contaminants found (arsenic ≠ hardness ≠ bacteria — they need different treatments).
7. **Acts**: drafts a ready-to-send complaint to the relevant water board, gated behind explicit human approval.
8. **Tracks & escalates**: remembers the complaint, follows up over days, and — if unresolved — drafts an RTI application. If many reports in one area share a contaminant, it drafts a collective complaint.

JUDGE SIGNAL — "CITATION RECEIPTS"

Make this the headline of the writeup and the money shot of the video. "Every threshold and every health flag carries a receipt back to its source, and a verifier agent blocks anything uncited." For a health application this is not a gimmick — it is the responsible-AI story Google DevRel actively rewards, and it visibly demonstrates the Day-4 "agent quality" discipline.

Why "Agents for Good" is the right home

This is squarely a public-good problem — safe drinking water — solved in a way that keeps a private individual's data private and hands power back to citizens and local bodies. It fits the track's spirit precisely: optimizing public health and supporting people who are otherwise unheard. (Per the rules, judges may move standout winners between tracks, so we optimize for fit and rigor, not for a prize that is identical swag across every track.)

Why Agents — Not a Chatbot

The single most important sentence in the video: **a chatbot can describe your water; only an agent can verify itself, file your complaint, and escalate it weeks later.**

Capability the problem demands	A single-prompt chatbot	A WaterWatch multi-agent system
Read a messy lab report	One-shot, error-prone, no structure guarantee	Dedicated Parser agent with structured output + confidence
Use authoritative limits	Recalls from memory — may hallucinate	Tool call to an MCP server holding BIS 10500 — grounded
Avoid unsafe claims	No self-check	Verifier (LoopAgent) blocks any uncited statement
Take real action	Produces text the user must act on	Drafts + tracks the complaint, gated by human approval
Remember across weeks	Stateless	Firestore-backed memory; follows up and escalates
See civic patterns	Cannot — one conversation	A2A aggregation agent detects pincode-level clusters

This maps directly to the course's own framing of an agent: a system that uses an LLM as its reasoning core but goes further — it plans over multiple steps, calls tools to take action, holds memory, coordinates specialists, and verifies its own work. WaterWatch is a textbook demonstration of every one of those properties, applied to a problem where each property is genuinely necessary rather than decorative.

THE "DECORATIVE VS NECESSARY" TEST

Most capstones bolt on multi-agent structure because the rubric asks for it. In WaterWatch, remove any agent and the system genuinely fails: drop the Verifier and it becomes unsafe; drop the Watchdog and it stops being accountable; drop the MCP layer and it starts guessing. **Necessity is the strongest possible argument to a judge.**

System Architecture

An ADK orchestrator coordinates six specialized agents. Each is small, single-purpose, and reliable — the opposite of one giant prompt. The specialists are exposed to the root agent as `AgentTools`, and ADK's workflow primitives (`SequentialAgent` , `ParallelAgent` , `LoopAgent`) wire the flow.

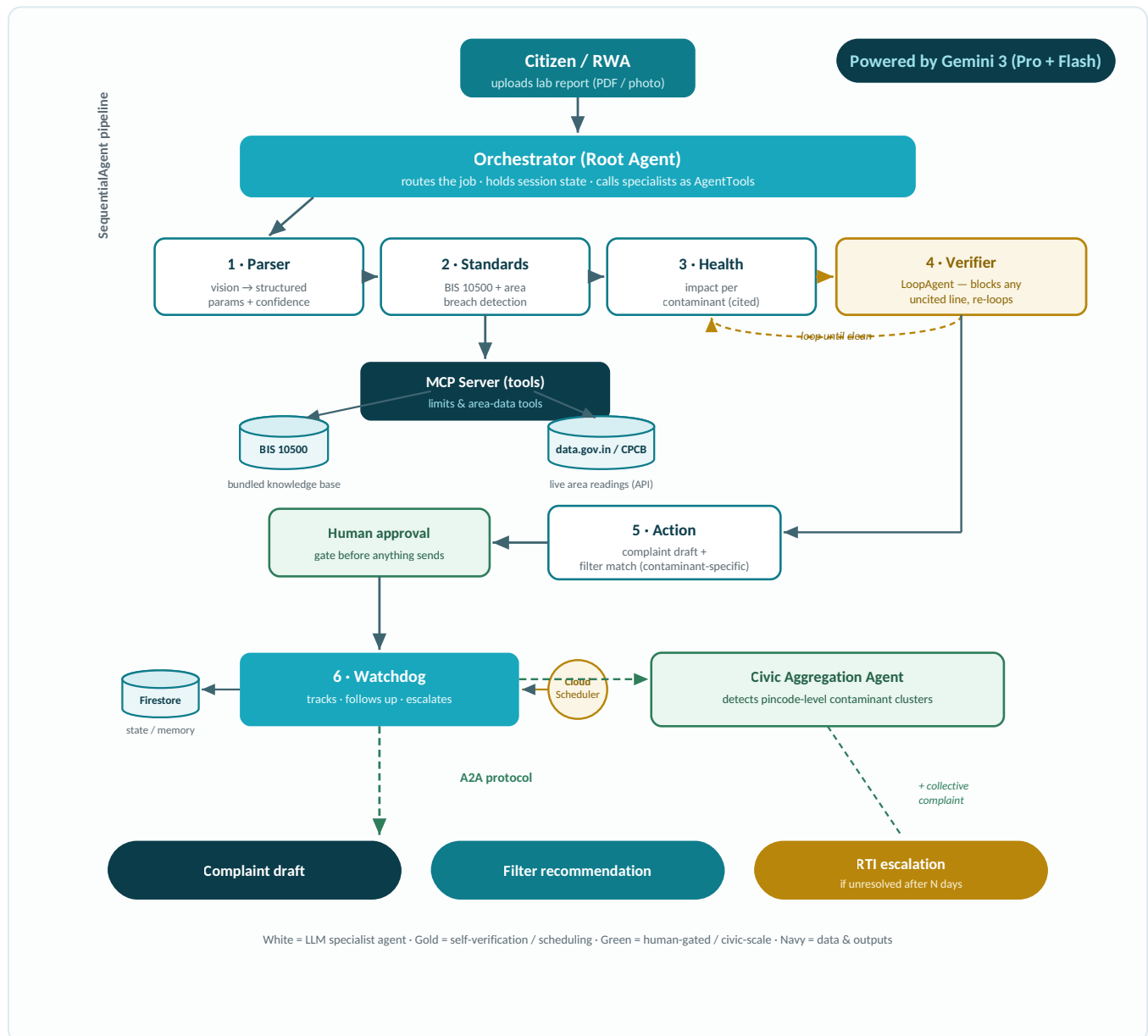


Figure 1 — WaterWatch agent pipeline. The orchestrator runs Parser → Standards → Health as a sequence, the Verifier loops until every claim is cited, the Action agent drafts behind a human gate, and the Watchdog persists state and escalates, talking to a Civic Aggregation Agent over A2A.

The six agents in detail

1 Parser Agent

Input: uploaded report (PDF/image) · **Output:** structured JSON of parameters + per-field confidence · **Tech:** Gemini 3 multimodal

Reads the document directly — no separate OCR step — and emits a typed parameter set. Low-confidence fields are surfaced for the user to confirm rather than silently guessed.

2 Standards Agent

Tools: MCP `get_bis_limit`, `get_area_readings` · **Output:** breach list with magnitudes

For each parameter, calls the MCP server for the BIS 10500 acceptable/permissible limit and the area's official readings. Its instructions forbid stating any limit not returned by a tool — this is where hallucinated thresholds are designed out.

3 Health-Impact Agent

Source: grounded WHO/BIS health-effects knowledge base · **Output:** per-contaminant impact, each with a citation

Translates "fluoride 1.8 mg/L (limit 1.0)" into a plain-language, cited health explanation (e.g. dental/skeletal fluorosis risk). Never free-form medical advice — only sourced statements.

4 Verifier Agent — the differentiator

Primitive: ADK `LoopAgent` (critic pattern) · **Output:** verified report or a re-loop instruction

Re-reads the assembled report and rejects any sentence lacking a citation receipt, sending it back for correction until the whole output is clean. This is the AgentOps / self-evaluation discipline from Day 4, embedded into runtime.

5 Action Agent

Output: municipal complaint draft + contaminant-specific filtration recommendation · **Gate:** human approval before send

Drafts a complete, correctly-addressed complaint and matches treatment to the actual contaminants found — arsenic needs activated alumina/RO, hardness needs a softener, bacteria need UV/UF. Nothing is dispatched without the user's explicit confirmation.

6 Watchdog Agent (+ Civic Aggregation via A2A)

Memory: Firestore · **Trigger:** Cloud Scheduler · **Protocol:** A2A to aggregation agent

Persists each complaint, follows up on a schedule, and drafts an RTI escalation if it stays unresolved past a threshold.

When several reports in one pincode share a contaminant, it coordinates over A2A with a Civic Aggregation Agent to draft a collective complaint — turning isolated cases into a public-health signal.

The MCP Data Layer

The data layer is exposed as a Model Context Protocol (MCP) server — the Day-2 concept — so the grounding tools are clean, discoverable, and reusable. This is what makes WaterWatch grounded rather than guessing.

Tools the MCP server exposes

Tool	Returns	Backed by
<code>get_bis_limit(param)</code>	Acceptable + permissible limit for a parameter	Bundled BIS 10500 table
<code>evaluate_sample(params)</code>	Breach list with magnitude & severity	Bundled logic + BIS
<code>get_area_readings(pincodes)</code>	Recent official readings for the locality	data.gov.in / CPCB API
<code>match_filtration(contaminants)</code>	Cheapest treatment per contaminant set	Curated treatment KB
<code>health_effect(contaminant)</code>	Cited health impact summary	WHO/BIS references KB

Why a real data source matters

India's Open Government Data platform publishes water-quality datasets in the Water Resources sector with direct API access via an API key, and the Central Pollution Control Board (CPCB) runs the National Water Quality Monitoring Programme. That means `get_area_readings` is a genuine live tool call, not a hand-wave — and it lets the agent answer "has the municipality actually reported quality in my area recently?" with real data. Google Search grounding (Day 1) acts as a fallback for fresh local advisories not yet in the structured datasets.

RESILIENCE BUILT IN

Government APIs can be slow or inconsistent. The BIS limits and a last-known snapshot of area data ship *bundled* with the MCP server, so the core verdict never depends on a live call succeeding. A failed external fetch degrades to "area comparison unavailable" — it never blocks the safety analysis.

Contaminant → treatment mapping (a slice of the curated KB)

Contaminant found	Cheapest effective treatment
Arsenic (As)	Activated alumina / RO / arsenic-removal media
Fluoride (F)	Activated alumina / RO / bone-char
Nitrate (NO ₃ ⁻)	RO / ion exchange
Hardness (Ca, Mg)	Ion-exchange water softener
Bacteria / <i>E. coli</i>	UV / ultrafiltration / boiling / chlorination
High TDS	Reverse osmosis

This table is real domain knowledge, and it doubles as the evidence that the agent's recommendation is reasoned from the specific contaminants found — not a generic "buy an RO."

Technology Stack

Everything sits inside the Google ecosystem — which is both the right engineering call and a direct scoring advantage in a Google-judged hackathon.

Layer	Choice	Why
Agent framework	Agent Development Kit (ADK)	Native multi-agent orchestration, workflow primitives, eval & deploy built in
Reasoning model	Gemini 3 Pro + Flash (via Vertex AI)	Pro for parsing/reasoning, Flash for cheap sub-tasks; keeps cost and latency down
Tooling protocol	MCP server (custom)	Clean, discoverable, reusable grounding tools (Day 2)
Agent-to-agent	A2A protocol	Citizen agent ↔ Civic Aggregation agent (Day 5)
Runtime	Cloud Run (container)	Serverless, scale-to-zero, simple reproducible deploy
State / memory	Firestore (Native)	Complaints, tracking, pincode index, session persistence (Day 3)
File storage	Cloud Storage	Uploaded reports, short lifecycle auto-delete for privacy
Secrets	Secret Manager	API keys never touch the codebase
Scheduling	Cloud Scheduler	Daily watchdog escalation checks
Observability	Cloud Logging + Cloud Trace	Traces and metrics for the eval story (Day 4)
CI / image	Cloud Build + Artifact Registry	Reproducible container builds
Build environment	Google Antigravity (agentic IDE)	Build it <i>with</i> agents; capture Artifacts for the video
Frontend	Lightweight web UI on Cloud Run (Firebase Hosting later)	One deployable for the MVP; split out only if needed

JUDGE SIGNAL — BUILD IT IN ANTIGRAVITY, THEN SHOW IT

Antigravity lets agents plan, execute, and verify across editor, terminal, and browser, and it produces Artifacts (implementation plans, task lists, screenshots, browser recordings). Drop one of those Artifact walkthroughs into the demo video — it ticks the "Antigravity" rubric line concretely, which most teams will only claim, not show.

Live GCP Deployment Architecture

You want it live in GCP — here is the real topology and a reproducible path. Deploying is not required for judging, but a working public deploy plus reproducible docs is a strong, visible signal, and it is genuinely achievable for a two-person team on this stack.

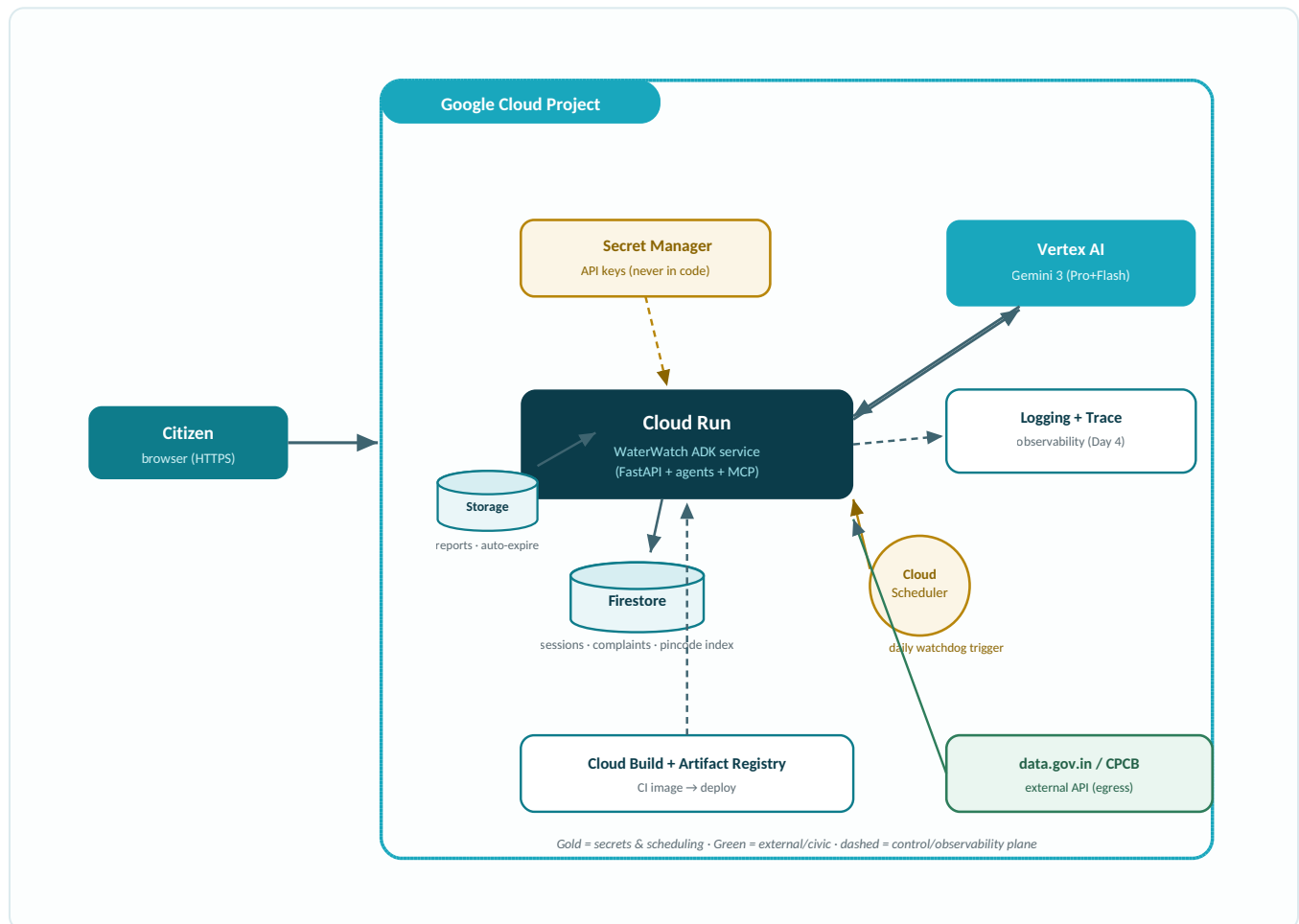


Figure 2 — GCP deployment topology. A single Cloud Run service hosts the ADK app and MCP server; Firestore holds state, Storage holds reports, Secret Manager holds keys, Scheduler drives the watchdog, and Logging/Trace provide the observability needed for the evaluation story.

Reproducible deploy (checklist, not a calendar)

- Provision:** enable APIs (`run` , `aipatform` , `firestore` , `secretmanager` , `cloudscheduler` , `storage` , `cloudbuild` , `artifactregistry` , `logging` , `cloudtrace`).
- Create resources:** Firestore (Native), a Storage bucket with a lifecycle rule, and the required secrets (e.g. the `data.gov.in` key).
- Build & deploy:** containerize and ship — `adk deploy cloud_run` for the fast path, or `gcloud run deploy --source .` (Vertex AI Agent Engine is the managed alternative if you want zero infra.)
- Wire the watchdog:** a Cloud Scheduler job hits a protected Cloud Run endpoint daily to run escalation checks.
- Lock down IAM:** a dedicated least-privilege service account per service; the runtime account gets only `aipatform.user` , Firestore, Storage, and secret-access roles it needs.

Every one of these is captured as commands in the README so a judge can reproduce the deployment — which is exactly what the Technical Implementation criterion asks for.

Security & Responsible-AI Design

Security is a named rubric concept and a Day-1 theme (identity and constrained policies). For a health app handling personal data, it is also simply the right thing to do — and WaterWatch treats it as architecture, not an afterthought.

Concern	How WaterWatch handles it
Secrets	All API keys in Secret Manager, injected at runtime. Nothing in the repo. (The rules flag "no API keys in code" twice — we honor it by design.)
Least privilege	Per-service IAM service accounts with only the roles each needs; no broad project-owner credentials in the runtime.
Unsafe autonomy	The agent <i>drafts</i> ; a human <i>approves</i> . No complaint or RTI is ever dispatched autonomously — a hard human-in-the-loop gate.
PII (name, address on reports)	Reports stored in Storage with short auto-delete lifecycle; PII kept out of logs; only the structured, non-identifying parameters drive the analysis.
Prompt injection via documents	Uploaded document text is treated as untrusted data, never as instructions; the verifier and tool-only grounding contain any injected content.
Hallucinated health/limit claims	Grounding constraint (tool-sourced limits only) + Verifier loop that blocks uncited statements. Safety enforced by construction.
Abuse / cost	Cloud Run scale-to-zero, request limits, and Gemini Flash for cheap sub-tasks bound both exposure and spend.

THIS IS TWO RUBRIC POINTS AT ONCE

The "grounding + verifier" mechanism is simultaneously your **Security** story (no unsafe output) and your **Agent Quality** story (Day 4 evaluation). One elegant mechanism, two scored concepts — that efficiency is itself a sign of good design to a judge.

How Course Days 1–5 Build This Project

Each day of the intensive maps onto a concrete, load-bearing part of WaterWatch. This is the section to mirror in your README, because it proves to judges that the build is a deliberate application of the course — not a project that happens to use Gemini.

Day	What the day taught	Where WaterWatch uses it
Day 1 Intro to Agents	Foundations of agents vs. LLM apps; first agent and first multi-agent system in ADK with Gemini; grounding answers with Google Search.	The entire orchestrator + six-specialist design; Gemini multimodal report parsing; Google Search grounding as the fallback for fresh local advisories.
Day 2 Tools & MCP	How agents take action via tools, and how MCP makes tools discoverable and interoperable.	The custom MCP server wrapping BIS 10500 limits, the data.gov.in/CPCB area API, the treatment KB, and the health-effects KB; specialists exposed as AgentTools.
Day 3 Context: Sessions & Memory	Short- and long-term memory; maintaining context across multi-turn, long-running tasks.	Firestore-backed complaint tracking over days/weeks, follow-up state, the pincode-level aggregation memory, and per-user session continuity.
Day 4 Agent Quality	Evaluation strategies, observability, logging, and tracing for reliable agents.	The Verifier LoopAgent (runtime self-check), a labelled eval set of sample reports with known verdicts, and Cloud Trace/Logging dashboards over agent runs.
Day 5 Prototype to Production	Deploying and scaling agents; building true multi-agent systems with the A2A protocol.	Cloud Run / Vertex deployment, the Cloud Scheduler watchdog, and A2A between the citizen agent and the Civic Aggregation agent.

WHY THIS TABLE IS WORTH POINTS ON ITS OWN

Most submissions touch two or three days and gesture at the rest. WaterWatch uses **all five**, each in a way that is necessary to the product. Putting this table in the writeup and README makes that legible at a glance — judges reward projects that visibly close the loop on the curriculum.

Winning Strategy: Scoring the Rubric

The rubric is 100 points across two categories. Here is exactly how WaterWatch earns in each, and where to concentrate effort.

Required concepts — you need 3, you hit 6

- ✓ Multi-agent (ADK)
- ✓ MCP Server
- ✓ Security features
- ✓ Deployability (Cloud Run)
- ✓ Antigravity (build + demo)
- ✓ Agent skills / CLI (ADK CLI + ADK Skills)

Hitting all six (when only three are required) is itself a flex — say so explicitly in the writeup.

Category 1 — The Pitch (30 pts)

Criterion	Pts	How we win it
Core concept & value	10	"Verifiable civic-action agent" — a clear, novel, high-impact use of agents where the agentic design is central, not bolted on.
YouTube video	10	Tight 5-minute cut with the verifier "catching itself" as the money shot (see Section 11).
Writeup	10	Lead with citation receipts; include both architecture diagrams and the Day 1–5 table; ≤2,500 words.

Category 2 — The Implementation (70 pts)

Criterion	Pts	How we win it
Technical implementation	50	Genuine multi-agent architecture, a real MCP server, clever tool use, clean commented code, and the eval harness almost nobody builds.
Documentation	20	README covering problem, solution, architecture, setup, diagrams, course mapping, and security (skeleton in Appendix A).

The three outlier moves

1 • CITATION RECEIPTS

Verifiable by design. Every threshold and health claim is sourced; the verifier blocks anything uncited. This is the trust story for a health app.

3 • ACTUATION + RTI ESCALATION

It doesn't just advise — it files, tracks, and escalates with an RTI when ignored. An India-specific, high-"wow" action no travel-planner clone will have.

2 • EVALUATION HARNESS

A labelled set of sample reports with known-correct verdicts, scored with pass/fail + traces. The Day-4 discipline that lifts you above the field.

THE SENTENCE JUDGES REPEAT

"An agent that files an RTI when the water board ignores you." Make sure that line lands in both the video and the writeup — it is what gets remembered in the room.

Demo & 5-Minute Video Plan

A third of the score lives in the pitch, and most technical teams under-invest here. Treat the video as a product. Keep it at or under 5 minutes, publish to YouTube, and attach a strong cover image.

Time	Beat	What's on screen
0:00-0:30	Problem	A real, unreadable lab report. "A family can't read this. A chatbot might lie about it."
0:30-1:00	Why agents	One sentence: a chatbot describes; only an agent verifies, files, and escalates.
1:00-2:00	Architecture	Figure 1 + Figure 2, narrated in plain language — orchestrator, MCP grounding, verifier loop, watchdog/A2A.
2:00-4:00	Live demo	Upload → verdict with receipts → the verifier rejecting a bad claim (the money shot) → complaint draft → RTI escalation. Include an Antigravity Artifact walkthrough.
4:00-4:30	The build	ADK + MCP + A2A + GCP, built in Antigravity. Fast montage.
4:30-5:00	Impact + future	The civic-scale vision (Section 14) and a clear "what this means for safe water."

THE SINGLE MOST IMPORTANT 10 SECONDS

Show the agent *catching itself* — the verifier blocking an uncited claim and forcing a correction. That one moment proves the entire trust thesis in a way no slide can. Rehearse it so it's crisp.

Team Split (2 builders)

A clean division of labor with one shared seam (the agent ↔ platform contract). Pair on architecture and the demo script; split everything else.

Role	Owns	Deliverables
Builder A — Agent / AI	ADK multi-agent core, MCP server, Verifier, eval harness, the knowledge bases (BIS limits, treatment, health effects)	Working agent pipeline + eval results + traces
Builder B — Platform / Product	GCP deploy (Cloud Run, Firestore, Storage, Secret Manager, Scheduler, Trace), security/IAM, frontend, README, writeup, video	Live deployment + docs + the 5-min video

Shared seam: agree the request/response contract and Firestore schema early so the two halves integrate without friction. Both review the final demo script.

Risks & Mitigations

Risk	Mitigation
Gov API downtime / inconsistency	Bundle BIS limits + a last-known data snapshot; degrade gracefully — never block the safety verdict.
Parsing errors on diverse lab formats	Per-field confidence + user confirmation step; few-shot examples for common Indian lab layouts.
Hallucinated limits / health claims	Tool-only grounding + Verifier loop (core design).
Wrong/auto-sent complaint	Hard human-approval gate before any dispatch.
PII leakage	Short-lived storage, redaction, no PII in logs.
Scope creep (2 people, ~13 days)	Ship the core loop first (parse → verify → complaint); Watchdog + A2A are clearly-scoped stretch goals.
Cost	Scale-to-zero, Flash for cheap sub-tasks, Pro only where it earns its keep.

Advanced Future Roadmap

Where WaterWatch goes after the capstone. Put a condensed version of this in the video's closing 30 seconds — judges reward a project with a credible path beyond the prototype.

Adoption & reach

- **Vernacular support** — Hindi, Gujarati, Tamil and more, in text and voice, so RWAs and panchayats can actually use it.
- **WhatsApp interface** — meet users where they are: send a photo of the report, get a verdict and a complaint draft back in chat.
- **Officials' dashboard** — give municipal engineers an aggregated heatmap of complaints and contaminant clusters, closing the loop on the government side.

Depth & intelligence

- **Live IoT sensors** — integrate low-cost TDS/turbidity sensors for continuous monitoring and anomaly alerts, not just one-off tests.
- **Predictive contamination** — forecast seasonal/monsoon contamination per region from historical CPCB data, warning before a test is even taken.
- **Tamper-evident audit trail** — hash-chained complaint records so "resolved" claims can't be quietly reversed; optional public transparency portal with a per-ward water-accountability index.

Platform & ecosystem

- **One engine, many domains** — the "standard + grounding + verification + actuation" pattern generalizes directly to *air quality* (CPCB AQI), soil, and food adulteration. WaterWatch becomes the first module of a broader **CivicGuard** platform.
- **An A2A agent marketplace** — connect to a lab-booking agent (schedule a retest), a filter-vendor agent (compare and order treatment), and an NGO/community agent (mobilize a neighborhood). This is the natural, ambitious end-state of the A2A foundation laid in the capstone.
- **Privacy-preserving on-device parsing** — run sensitive PII extraction on-device (e.g. Gemini Nano), sending only de-identified structured data to the cloud.

THE STRATEGIC NARRATIVE

WaterWatch is not "a water app" — it's the first proof that the verify-then-act agent pattern can make civic institutions accountable to ordinary people. That framing is what turns a capstone into a portfolio-defining project.

README.md skeleton

Drop this into the repo root and fill it in. It is structured to satisfy the 20-point Documentation criterion exactly — problem, solution, architecture, setup, diagrams, course mapping, and security — and to include the Day 1–5 justification you asked for.

```
# WaterWatch – the agent that holds your water board accountable

> Verifiable, civic-action multi-agent system for drinking-water safety.
> Built for the Google x Kaggle 5-Day AI Agents Intensive (Vibe Coding) capstone.
> Track: Agents for Good.

## 1. The Problem
- Water lab reports are unreadable to residents / RWAs / panchayats.
- A plain chatbot can hallucinate a safe limit – dangerous for a health decision.
- No clear, trackable path to a municipal complaint, and no civic-scale signal.

## 2. The Solution
Upload a report -> verified safety verdict (with citation receipts) ->
contaminant-specific filter advice -> drafted complaint -> tracked -> RTI escalation.
Core principle: the agent never asserts a number or health claim it cannot cite.

## 3. Architecture
- Orchestrator (root) + 6 specialists, specialists exposed as AgentTools.
- Parser -> Standards -> Health (SequentialAgent); Verifier (LoopAgent) blocks
  uncited claims; Action drafts behind a human-approval gate; Watchdog persists
  state, escalates, and talks to a Civic Aggregation agent over A2A.
- Grounding via a custom MCP server (BIS 10500 + data.gov.in/CPCB + treatment KB).
- (Insert architecture diagram images here: docs/agent-pipeline.png, docs/gcp.png)

## 4. Tech Stack
ADK · Gemini 3 (Vertex AI) · MCP · A2A · Cloud Run · Firestore · Cloud Storage ·
Secret Manager · Cloud Scheduler · Cloud Logging + Trace · built in Antigravity.

## 5. How Course Days 1–5 Map to This Project
- Day 1 (Agents/ADK): orchestrator + multi-agent design; Gemini parsing; Search grounding.
- Day 2 (Tools/MCP): the MCP server and AgentTools.
- Day 3 (Memory): Firestore complaint tracking + pincode aggregation.
- Day 4 (Quality): Verifier loop + eval harness + Cloud Trace.
- Day 5 (Production): Cloud Run/Vertex deploy + Scheduler watchdog + A2A.

## 6. Setup & Deploy
```
1. Auth + enable APIs (run, aiplatform, firestore, secretmanager,
cloudscheduler, storage, cloudbuild, artifactregistry, logging, cloudtrace)
2. Create Firestore (Native), a Storage bucket (+ lifecycle rule), and secrets
3. Deploy: adk deploy cloud_run (or: gcloud run deploy --source .)
4. Create the Cloud Scheduler job -> protected /watchdog endpoint
5. Apply least-privilege IAM per service account
```
Local dev: `adk web` (or `adk run`) with a `.env` that is never committed.

## 7. Security
Secrets in Secret Manager (no keys in code) · least-privilege IAM ·
human-in-the-loop before any complaint/RTI sends · PII auto-expires ·
uploaded docs treated as untrusted (prompt-injection safe) · grounded, verified output.
```

8. Demo

YouTube: (link) · Live app: (link)

9. License

(Choose one — e.g. Apache-2.0)

BEFORE THE REPO GOES PUBLIC

Scrub for keys and secrets. Confirm `.env`, service-account JSON, and any notebook outputs containing tokens are git-ignored and absent from history. The rules call this out twice — a leaked key can sink an otherwise winning entry.

Repository structure

A clean, conventional ADK layout that reads well to a judge and keeps the two builders' work cleanly separated.

```

waterwatch/
├── README.md           # the doc above
├── pyproject.toml      # deps (adk, mcp, google-cloud-*)
├── .env.example        # keys documented, NOT real values
├── Dockerfile         # Cloud Run container
├── docs/
│   ├── agent-pipeline.png # Figure 1
│   └── gcp-topology.png   # Figure 2
├── waterwatch_agent/
│   ├── agent.py         # root orchestrator
│   ├── sub_agents/
│   │   ├── parser.py    # 1 · multimodal extraction
│   │   ├── standards.py # 2 · BIS + area, tool-grounded
│   │   ├── health.py    # 3 · cited health impact
│   │   ├── verifier.py  # 4 · LoopAgent critic
│   │   ├── action.py    # 5 · complaint + filter match
│   │   └── watchdog.py  # 6 · tracking + RTI + A2A
│   └── prompts/         # instruction templates per agent
├── mcp_server/
│   ├── server.py        # MCP tools (Day 2)
│   └── data/
│       ├── bis_10500.json # bundled standard (ground truth)
│       ├── treatment_kb.json # contaminant -> filtration
│       └── health_kb.json  # contaminant -> cited effects
├── eval/
│   ├── dataset/         # labelled sample reports
│   └── run_eval.py       # pass/fail + traces (Day 4)
├── deploy/
│   ├── cloudbuild.yaml
│   └── scheduler.yaml   # watchdog cron
└── frontend/           # lightweight upload + results UI
  
```

WaterWatch — build it grounded, make it act, prove it works.
Verify, then escalate. That's the whole idea.